

HLD Case Study 2: Ticketing & Hotel Reservation System (Ticketmaster / Booking.com)

Key Focus Areas: Concurrency control, distributed locking (Redis Redlock), preventing double-booking, idempotency, and temporary hold expiration.

1. Problem Statement & Requirements

Design a global ticketing platform for high-demand concerts where 50,000 seats sell out within seconds.

Functional Requirements:

1. Search events, venues, and view available seats in real-time.
2. Select and temporarily hold seats for 10 minutes while user completes checkout.
3. Confirm booking upon successful payment capture.
4. Auto-release seats back to the public pool if payment times out.

Non-Functional Requirements:

1. **Strict Zero Double-Booking:** Never sell the same seat to two customers under any failure scenario.
 2. High availability for browsing; strong consistency for booking.
-

2. High-Level Architecture Diagram

flowchart TD

Client["Client App"] → CDN["CDN (Cached Event Info)"]

Client → ALB["Application Load Balancer"]

ALB → Gateway["API Gateway"]

Gateway → SearchSvc["Search Service (Elasticsearch)"]

Gateway → BookingSvc["Booking Service"]

BookingSvc → RedisLock["Redis Cluster (Distributed Locks & Seat Holds with TTL)"]

BookingSvc → RDBMS["PostgreSQL (ACID Source of Truth)
(Row-level locks / Read-C

BookingSvc → PaymentSvc["Payment Service (Stripe Gateway)"]

BookingSvc → Kafka["Kafka: booking_events"]

Kafka → TimeoutWorker["Hold Expiration Worker"]

3. Distributed Locking & The Seat Reservation Lifecycle

```
sequenceDiagram
    autonumber
    actor User as Customer
    participant Svc as Booking Service
    participant Redis as Redis Lock Engine
    participant DB as PostgreSQL
    participant Pay as Payment Gateway

    User->>Svc: reserve_seats(show_id, seat_ids=[A1, A2])
    Svc->>Redis: SET lock:show_1:A1 user_123 NX EX 600
    alt Lock Acquired on all seats
        Redis->>Svc: Success (10-minute hold active)
        Svc->>DB: INSERT INTO reservations (status='HELD', expires_at=NOW()+10min)
        Svc->>User: 200 OK (Seats held; please pay within 10 mins)

        User->>Svc: checkout(reservation_id, payment_token)
        Svc->>Pay: charge(amount)
        alt Payment Success
            Pay->>Svc: 200 Paid
            Svc->>DB: UPDATE reservations SET status='CONFIRMED'; UPDATE seats SET status
            Svc->>Redis: DEL lock:show_1:A1
            Svc->>User: 201 Created (Tickets Confirmed!)
        else Payment Failed / Timeout
            Pay->>Svc: Declined
            Svc->>Redis: DEL lock:show_1:A1 (Immediately released back to pool!)
        end
    end
    else Lock Failed (Another user got there first)
        Redis->>Svc: Key already exists!
        Svc->>User: 409 Conflict: Seats currently held by another user.
    end
end
```

4. Database Schema (PostgreSQL with Optimistic Locking)

```
CREATE TABLE seats (  
    seat_id UUID PRIMARY KEY,  
    show_id UUID NOT NULL,  
    seat_number VARCHAR(10) NOT NULL,  
    status VARCHAR(20) NOT NULL, -- AVAILABLE, HELD, BOOKED  
    version INT NOT NULL DEFAULT 1, -- Optimistic locking counter  
    UNIQUE(show_id, seat_number)  
);  
  
-- Atomic SQL reservation query preventing race conditions:  
UPDATE seats  
SET status = 'HELD', version = version + 1  
WHERE seat_id = 'c4b8- ... ' AND status = 'AVAILABLE' AND version = 1;
```